

Business Requirements Document (BRD)

AI JIRA Story Generator

Project	AI JIRA Story Generator ("techTitans")
Document Owner	techTitans
Status	Draft
Date	2026-07-22
Related Artifacts	AI-JIRA-Story-Generator-Design.docx, AI-JIRA-Story-Generator-Business-Deck.pptx

1. Executive Summary

Business Analysts and Product Managers spend significant time converting raw meeting discussions (requirements-gathering calls, sprint planning sessions, stakeholder interviews) into structured, sprint-ready JIRA items. This process is manual, inconsistent in quality, and slow — especially when transcripts are long, contain multiple languages, or mix requirements with risks, action items, and open questions.

The **AI JIRA Story Generator** is an internal web application that automates this conversion. A user pastes or uploads a meeting transcript (text, .docx, or .pdf); the system redacts personally identifiable information (PII), normalizes the language to English, extracts discrete requirements, grounds them against known internal system context, asks targeted clarifying questions where facts are missing, drafts type-specific JIRA items (Epics, Stories, Bugs), scores each against quality criteria (INVEST-style), and — once quality thresholds are met — creates or updates the corresponding issues directly in JIRA Cloud.

The goal is to reduce the time BAs/PMs spend writing tickets, improve consistency and completeness of ticket quality, and provide a privacy-conscious, auditable pipeline from "conversation" to "backlog item."

2. Business Context / Problem Statement

- Meeting-to-backlog translation is currently a manual, repetitive task performed by BAs/PMs after every requirements or planning session.
- Ticket quality varies by author — acceptance criteria, business value statements, and reproduction steps are often incomplete, leading to rework and back-and-forth during sprint refinement.
- Meetings frequently include multiple speakers, filler speech, ASR (speech-to-text) artifacts, and occasionally non-English or mixed-language content, all of which slow down manual note-taking.
- Meeting transcripts may contain sensitive personal information (names, emails, phone numbers) that should not be sent to third-party AI services in raw form.
- Teams need generated tickets to be grounded in the actual systems being discussed (e.g., correctly distinguishing a payments-gateway story from an onboarding-system story) rather than generic, context-free descriptions.

3. Business Objectives

ID	Objective
OBJ-1	Reduce the time required to convert a meeting transcript into JIRA-ready tickets from hours to minutes.
OBJ-2	Improve consistency and completeness of generated tickets by enforcing structured, type-specific templates and automated quality scoring.
OBJ-3	Ensure no unredacted personal data is transmitted to external AI services.
OBJ-4	Support meetings conducted in multiple languages without requiring manual translation.
OBJ-5	Ground generated tickets in the correct internal system/application context to reduce ambiguity and rework.
OBJ-6	Give BAs/PMs a review-and-refine step before tickets are created in JIRA, preserving human control over what enters the backlog.

4. Scope

4.1 In Scope

- Ingestion of meeting transcripts via pasted text or file upload (.txt, .md, .docx, .pdf).
- Automated transcript cleanup (timestamps, filler words, ASR noise removal).
- PII detection and redaction prior to any external AI call.
- Language detection and normalization to English.
- Extraction of discrete requirements, risks, action items, and open questions from a transcript.
- Matching of discussion content to known internal applications/systems, using a maintained knowledge base, to ground generated tickets in real context.
- Interactive clarifying questions when a required fact is missing, with a bounded number of questions per item and a response timeout.
- Drafting of three JIRA issue types — **Epic**, **Story**, **Bug** — each with a type-specific schema (e.g., Stories include user story + acceptance criteria + story points; Bugs include repro steps + severity; Epics include goal + business value + success criteria).
- Automated quality validation and scoring (0–100) of each generated item, with automatic refinement passes when below a configured threshold.
- On-demand item actions: improve wording, expand acceptance criteria, generate test cases, generate risk analysis.
- Conversational review UI allowing the user to revise, regenerate, or scope generation (e.g., "just the epic," "bugs only") before committing to JIRA.
- Direct creation and update of issues in a configured JIRA Cloud project via the JIRA REST API, including a pre-create payload preview.
- Configurable auto-create behavior: items scoring above a quality threshold may be created in JIRA automatically; items below the threshold are held for manual review.
- Session history (list, rename, favorite, delete) so prior transcripts/generations can be revisited.
- Export of generated items as JSON, CSV, or Markdown.
- Light/dark theme UI.

4.2 Out of Scope (current version)

- User authentication, accounts, or role-based access control.
- Multi-user concurrency / hosted multi-tenant deployment (current architecture is single-process, single-user, local).
- Support for JIRA issue type "Task" (defined in the data model but not currently generated).
- Integration with JIRA products other than JIRA Cloud (e.g., JIRA Server/Data Center), or with non-JIRA issue trackers.
- Support for LLM providers other than Anthropic Claude.
- Automated test coverage of the pipeline (currently none exists).
- Secrets management beyond a local `.env` file.

5. Stakeholders

Role	Interest
Business Analysts / Product Managers	Primary users; author and review generated tickets before they reach JIRA.
Scrum Masters / Delivery Leads	Consumers of consistent, well-formed backlog items during sprint planning/refinement.
Engineering Teams	Consumers of Bugs/Stories with clear, testable acceptance criteria and reproduction steps.
Compliance / Data Privacy	Interested in assurance that PII is not sent to external AI services unredacted.
IT/Security	Interested in credential handling, hosting model, and access control before any broader rollout.

6. Functional Business Requirements

ID	Requirement	Priority
BR-1	The system shall allow a user to submit a meeting transcript as pasted text or as an uploaded <code>.txt</code> , <code>.md</code> , <code>.docx</code> , or <code>.pdf</code> file.	Must
BR-2	The system shall remove timestamps, filler words, and transcription artifacts from the raw transcript before further processing.	Must
BR-3	The system shall detect and redact personally identifiable information (e.g., names, emails, phone numbers, financial identifiers) before any transcript content is sent to an external AI service.	Must
BR-4	The system shall detect the transcript's language and normalize non-English or mixed-language content into English while preserving speaker labels, order, and technical/product terms.	Must

ID	Requirement	Priority
BR-5	The system shall extract discrete requirements, risks, action items, and open questions from the processed transcript.	Must
BR-6	The system shall match extracted content against a maintained catalog of known internal applications/systems and incorporate the relevant system context into generated tickets.	Should
BR-7	The system shall ask the user targeted clarifying questions when information required to complete a ticket is missing, up to a configurable maximum per item, and shall time out gracefully if unanswered.	Should
BR-8	The system shall generate JIRA items in the following types, each with its own required fields: Epic (goal, business value, success criteria), Story (user story, acceptance criteria, story points), Bug (steps to reproduce, expected/actual result, severity).	Must
BR-9	The system shall score each generated item against defined quality criteria and display the score to the user.	Must
BR-10	The system shall automatically attempt to refine an item that scores below the configured quality threshold, up to a configured maximum number of attempts.	Must
BR-11	The system shall allow the user to review, edit intent via conversational instructions, regenerate, or scope (e.g., limit to specific item types) generated items before creating them in JIRA.	Must
BR-12	The system shall allow the user to request supplementary content per item on demand: reworded text, expanded acceptance criteria, generated test cases, or a risk analysis.	Should
BR-13	The system shall create or update issues in a configured JIRA Cloud project via the JIRA REST API, mapping generated fields to native JIRA fields (summary, description, issue type, priority, labels, story points where applicable).	Must
BR-14	The system shall allow the user to preview the exact payload that will be sent to JIRA before an item is created.	Should

ID	Requirement	Priority
BR-15	The system shall support an auto-create mode where items meeting a configurable quality bar are pushed to JIRA automatically, and items below the bar are held for manual review.	Should
BR-16	The system shall retain a history of past sessions (transcript, generated items, logs, JIRA results) that a user can revisit, rename, favorite, or delete.	Should
BR-17	The system shall allow a user to export the generated items of a session in JSON, CSV, or Markdown format.	Could

7. Non-Functional Requirements

ID	Requirement	Notes
NFR-1	The system shall never transmit unredacted personal data to an external AI provider.	Currently enforced by ordering PII redaction before any LLM call.
NFR-2	The system shall bound concurrent AI calls during processing to avoid exceeding provider rate limits.	Currently a configurable concurrency limit.
NFR-3	The system shall retry transient failures (network errors, provider rate limiting, JIRA 5xx errors) with backoff before surfacing an error to the user.	
NFR-4	Generated ticket quality scoring and thresholds shall be configurable without code changes.	Currently via environment configuration.
NFR-5	[Gap — recommended for hardening] The system should support authentication and access control before being deployed beyond a single trusted user/local environment.	No auth currently exists; anyone with network access to the app has full access to all sessions and JIRA push capability.
NFR-6	[Gap — recommended for hardening] Credentials (AI provider key, JIRA API token) should be stored via a secrets manager rather than a plaintext local file, and the project should ship a template configuration file with placeholder values for onboarding.	No <code>.env.example</code> currently exists.
NFR-7	[Gap — recommended for hardening] The system should have automated test coverage for the transcript-to-ticket pipeline.	No automated tests currently exist; behavior is validated manually via sample transcripts.

ID	Requirement	Notes
NFR-8	[Gap — recommended for hardening] The persistence and session-coordination layer should support multi-process/multi-user operation if the tool is to be used by more than one person concurrently.	Current architecture uses an embedded single-file database and in-process coordination suited to a single running instance.

8. Assumptions

- Users have an existing JIRA Cloud project and valid API credentials to connect to it.
- Meeting transcripts are provided in a form the system can parse (plain text, or .docx/.pdf produced by standard tools).
- The internal application knowledge base used to ground tickets is kept up to date by the owning team; ticket grounding quality depends on this catalog's accuracy and coverage.
- Use of a third-party AI provider (Anthropic Claude) for text processing is an accepted and approved practice within the organization, subject to the PII-redaction control in BR-3/NFR-1.
- The tool is intended for internal, trusted use (BAs/PMs/delivery teams), not for external or public-facing deployment in its current form.

9. Constraints

- Single AI provider (Anthropic Claude) — no fallback provider is currently implemented.
- Single JIRA Cloud project/target per deployment configuration; no multi-project or cross-instance routing.
- No authentication layer — deployment should be restricted to a trusted network/environment until access control is added.
- Local, file-based data store — not designed for concurrent multi-user load at present.

10. Success Metrics

Metric	Description
Time-to-ticket	Average time from transcript submission to JIRA item creation, compared against manual baseline.
Ticket quality score	Average automated quality score of generated items, and proportion requiring manual edits after creation.
Clarification rate	Average number of clarifying questions asked per item (a proxy for how often source material is incomplete).
Auto-create rate	Proportion of generated items meeting the auto-create quality bar without manual intervention.
Adoption	Number of active users / sessions created per sprint cycle.

11. Risks

Risk	Impact	Mitigation
No authentication/access control	Any user on the network can view all sessions and push to JIRA	Restrict network access until an auth layer is added; treat as a blocker for any hosted/shared deployment
Plaintext credential storage	Leaked AI/JIRA credentials if the environment is compromised	Move to a secrets manager; rotate any credentials currently on disk; add <code>.env.example</code>
No automated tests	Regressions in extraction/drafting quality may go unnoticed	Introduce a test suite using existing sample transcripts as fixtures
Single-process architecture	Tool cannot reliably scale to concurrent multi-user use	Scope multi-user support as a follow-on phase if adoption grows beyond a single user/team
Reliance on a single AI provider	Service disruption or pricing changes from the provider directly affect availability/cost	Monitor provider dependency as a business continuity consideration

12. Glossary

Term	Definition
BA	Business Analyst
PM	Product Manager
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable — a quality heuristic for user stories
PII	Personally Identifiable Information
ADF	Atlassian Document Format — JIRA's rich-text description format
Epic / Story / Bug	JIRA issue types generated by the system, each with a distinct required field set
Session	A single transcript-to-tickets working conversation, tracked with its own history and state

Appendix A — Current Technical Snapshot (for context)

- **Backend:** Python / FastAPI, SQLite-based persistence, Anthropic Claude for all AI processing (a lighter model for language normalization, a stronger model for extraction/drafting/validation).
- **Frontend:** React + Vite + Tailwind CSS, single-page chat-style interface with transcript editor, workflow progress view, clarifying-question forms, and per-item review cards.
- **Integrations:** Anthropic Claude API; JIRA Cloud REST API v3 (Basic auth via email + API token).
- **Supporting capabilities:** PII redaction (Presidio/spaCy), file parsing (python-docx, pypdf), language detection (langdetect).

This appendix reflects the system as implemented at the time of writing and is provided for traceability between business requirements and the current build; it is not itself a requirement.